

UNIVERSIDAD — FACULTAD DE INGENIERÍA

Sistema de Votación Distribuido

Mascota de la Facultad

Documento de Decisiones de Diseño y Componentes de Sistemas Operativos y Redes

Elaborado por:

Sofia Moreno Quintero

Julian David Fuentes Granja

Asignatura:

Sistemas Operativos y Redes

Proyecto Final — 2026

Tabla de Contenido

1.	Descripción General del Proyecto	3
2.	Arquitectura del Sistema	4
3.	Decisiones de Diseño	5
4.	Componentes de Sistemas Operativos Aplicados	7
5.	Componentes de Redes Aplicados	10
6.	Diseño de la Base de Datos	13
7.	Flujo de Datos y Comunicación entre Servicios	14
8.	Seguridad y Manejo de Errores	15
9.	Conclusiones	16

1. Descripción General del Proyecto

El proyecto consiste en un **sistema distribuido de votación electrónica** desarrollado como proyecto final de la asignatura de Sistemas Operativos y Redes. Su propósito es permitir a los estudiantes de la facultad elegir la mascota oficial mediante una interfaz web intuitiva, garantizando que cada estudiante emita únicamente un voto válido.

El sistema implementa una arquitectura de tres capas totalmente contenerizada con Docker, donde cada capa cumple una responsabilidad específica dentro del ciclo de vida de una solicitud HTTP: presentación, lógica de negocio y persistencia de datos.

Candidatos disponibles:

- **Pavo Real:** Candidato 1 — elegante y colorido, símbolo de distinción académica.
- **Pato:** Candidato 2 — amigable y curioso, reflejo del espíritu estudiantil.
- **Ganso:** Candidato 3 — determinado y solidario, representante del trabajo en equipo.

Objetivos del sistema:

- Implementar un sistema web funcional con arquitectura de microservicios contenerizados.
- Demostrar conceptos de Sistemas Operativos: aislamiento de procesos, gestión de memoria, variables de entorno, IPC y persistencia.
- Aplicar conceptos de Redes: modelo cliente-servidor, HTTP/REST, reverse proxy, DNS interno Docker, puertos y sockets.
- Garantizar integridad del voto mediante restricciones de base de datos (PRIMARY KEY y FOREIGN KEY).
- Proveer resultados en tiempo real mediante polling periódico desde el cliente.

2. Arquitectura del Sistema

El sistema adopta una **arquitectura de tres capas** completamente contenerizada. Cada capa se ejecuta en su propio contenedor Docker con aislamiento de red, sistema de ficheros y recursos de CPU/memoria.

Diagrama de capas:

Capa	Tecnología	Puerto	Responsabilidad
Presentación	Nginx 1.27 Alpine	8080 → 80	Servir archivos estáticos (HTML/CSS/JS) y actuar como reverse proxy hacia la API
Lógica de negocio	Python 3.12 + Flask 3.1	5000 (interno)	Validar y procesar votos; consultar y retornar resultados en formato JSON
Persistencia	PostgreSQL 16 Alpine	5432 (interno)	Almacenar candidatos y votos; garantizar unicidad de voto por código

Flujo de red externo vs. interno:

El único punto de entrada expuesto al host es el puerto **8080** del contenedor Nginx. Tanto el backend Flask (5000) como PostgreSQL (5432) se comunican exclusivamente dentro de la red interna *voting_net* definida en Docker Compose, sin exponer puertos al exterior. Esto reduce la superficie de ataque y aplica el principio de mínimo privilegio de red.

Docker Compose orquesta el ciclo de vida completo: define el orden de arranque (db → backend → frontend), gestiona variables de entorno, monta volúmenes y configura los healthchecks que garantizan que los servicios dependientes no inicien antes de que sus dependencias estén listas.

3. Decisiones de Diseño

3.1 Contenerización completa con Docker

Se decidió contenerizar los tres servicios usando Docker en lugar de instalaciones directas en el sistema operativo huésped. Esta decisión ofrece reproducibilidad total del entorno: el sistema funciona de forma idéntica en cualquier máquina con Docker instalado, independientemente del sistema operativo. Cada contenedor encapsula sus dependencias, eliminando el problema clásico de 'funciona en mi máquina'. Adicionalmente, el uso de imágenes Alpine (Nginx:alpine, python:3.12-slim, postgres:16-alpine) minimiza la huella de disco y la superficie de ataque.

3.2 Arquitectura de tres capas desacopladas

Se separó deliberadamente la capa de presentación (Nginx), la lógica de negocio (Flask) y la persistencia (PostgreSQL) en contenedores independientes. Este desacoplamiento permite escalar cada capa de forma independiente, sustituir componentes sin afectar el resto del sistema y facilita el mantenimiento. Por ejemplo, el motor de base de datos podría cambiarse por MySQL sin modificar el frontend, o el backend podría reescribirse en Node.js sin alterar la BD.

3.3 Nginx como reverse proxy

En lugar de exponer Flask directamente, se interpuso Nginx como proxy inverso. Las peticiones a `/api/*` son redirigidas internamente hacia `http://backend:5000/`, mientras que el resto de rutas sirven archivos estáticos. Esta decisión unifica la interfaz de red del sistema en un único puerto (8080), oculta la topología interna al cliente, permite agregar SSL/TLS en un único punto, y aprovecha las capacidades de Nginx para servir archivos estáticos eficientemente (E/S asíncrona no bloqueante).

3.4 API REST sin estado (stateless)

La API Flask fue diseñada siguiendo los principios REST: sin estado de sesión en el servidor, recursos identificados por URL (`/votar`, `/resultados`), y comunicación mediante JSON. Esto facilita el escalado horizontal del backend (múltiples instancias pueden atender peticiones sin compartir estado) y hace la API predecible e interoperable con cualquier cliente HTTP.

3.5 Integridad del voto mediante restricciones de BD

La regla de negocio 'un código, un voto' se implementó como restricción de base de datos: el campo *codigo* en la tabla *votos* es PRIMARY KEY. Esta decisión delega la responsabilidad de integridad al motor de base de datos en lugar de la lógica de aplicación, garantizando que incluso peticiones concurrentes simultáneas no puedan duplicar votos gracias a las garantías ACID de PostgreSQL. El backend captura la excepción *UniqueViolation* y retorna HTTP 409 Conflict.

3.6 Resultados en tiempo real mediante polling

Se implementó un mecanismo de actualización de resultados mediante polling periódico (setInterval cada 5 segundos) desde el frontend JavaScript. Se consideraron alternativas como WebSockets o Server-Sent Events, pero el polling fue elegido por su simplicidad de implementación,

compatibilidad universal y suficiencia para un sistema de votación donde la latencia de actualización de 5 segundos es aceptable.

3.7 Variables de entorno para configuración sensible

Las credenciales de la base de datos (usuario, contraseña, nombre de BD) se inyectan mediante variables de entorno definidas en el archivo *.env*, excluido del repositorio mediante *.gitignore*. Esta práctica sigue el principio de los Doce Factores (The Twelve-Factor App) para separar configuración del código, permitiendo desplegar el mismo artefacto en entornos distintos (desarrollo, staging, producción) sin modificar el código fuente.

3.8 Volumen persistente para la base de datos

Los datos de votos se almacenan en un volumen Docker nombrado (*postgres_data*). Esta decisión garantiza que los votos emitidos sobrevivan reinicios del contenedor, actualizaciones de imagen y paradas planificadas del sistema, sin depender del sistema de ficheros efímero del contenedor.

3.9 Healthcheck y dependencias entre servicios

PostgreSQL incluye un healthcheck que verifica periódicamente su disponibilidad ejecutando *pg_isready*. El servicio backend declara *depends_on* con condición *service_healthy*, lo que garantiza que Flask no intente conectarse a la BD antes de que esté lista para aceptar conexiones. Sin este mecanismo, el backend podría fallar al arrancar si PostgreSQL aún está inicializando su cluster.

4. Componentes de Sistemas Operativos Aplicados

El proyecto aplica múltiples conceptos fundamentales de Sistemas Operativos, tanto a nivel de la infraestructura Docker como a nivel de la aplicación Flask y el servidor de base de datos. A continuación se detalla cada componente.

4.1 Procesos e Hilos

Cada contenedor Docker ejecuta exactamente un proceso principal (PID 1). En el contenedor del backend, el proceso es **python app.py**, que levanta el servidor Flask. Flask, al ejecutarse en modo de desarrollo o producción sin workers adicionales, gestiona las peticiones HTTP de forma síncrona en un único hilo. PostgreSQL, por su parte, crea un proceso *backend* independiente por cada conexión de cliente, lo que constituye un modelo clásico de **proceso por conexión**.

Nginx implementa un modelo de **I/O multiplexado** basado en el sistema de llamadas *epoll* (Linux) o *kqueue* (BSD). Un proceso maestro gestiona los worker processes, y cada worker atiende múltiples conexiones concurrentes sin bloquear, lo que lo hace altamente eficiente para servir archivos estáticos y proxying.

4.2 Namespaces y Grupos de Control (cgroups)

Docker utiliza las primitivas del kernel Linux para aislar contenedores:

- **PID namespace:** Cada contenedor tiene su propio árbol de procesos. El proceso Flask es PID 1 dentro del contenedor, aislado de los procesos del host.
- **NET namespace:** Cada contenedor dispone de su propia interfaz de red virtual (veth pair), tabla de rutas y stack TCP/IP. Esto permite que múltiples contenedores usen el mismo puerto (ej. todos en 80) sin conflicto.
- **MNT namespace:** El sistema de ficheros de cada contenedor está aislado mediante UnionFS (overlay2). Los cambios en el sistema de ficheros del contenedor no afectan al host.
- **UTS namespace:** Cada contenedor tiene su propio hostname (frontend, backend, db), lo que permite el descubrimiento de servicios por nombre dentro de la red Docker.
- **IPC namespace:** Aislamiento de memoria compartida y semáforos POSIX entre contenedores.
- **cgroups:** Docker puede limitar CPU, memoria RAM y E/S de disco por contenedor, garantizando que un servicio no monopolice los recursos del host.

4.3 Variables de Entorno como Mecanismo de Configuración del SO

El sistema operativo expone variables de entorno al proceso hijo en el momento de la creación (fork/exec). Docker Compose inyecta las variables definidas en el archivo *.env* al contenedor backend mediante el mecanismo *env_file*. El proceso Flask las lee mediante `os.environ['VARIABLE']`, sin necesidad de archivos de configuración adicionales. Esto aplica directamente el concepto de herencia de entorno en sistemas UNIX.

Variables de entorno utilizadas:

Variable	Descripción	Consumidor
----------	-------------	------------

DB_HOST	Hostname del servidor PostgreSQL (nombre del servicio Docker)	Flask / psycopg2
DB_PORT	Puerto de escucha de PostgreSQL (5432)	Flask / psycopg2
DB_NAME	Nombre de la base de datos (votacion)	Flask / psycopg2
DB_USER	Usuario de base de datos con permisos INSERT/SELECT	Flask / psycopg2
DB_PASSWORD	Contraseña del usuario de base de datos	Flask / psycopg2
POSTGRES_DB	Nombre de la BD a crear al inicializar PostgreSQL	PostgreSQL container
POSTGRES_USER	Superusuario inicial de PostgreSQL	PostgreSQL container
POSTGRES_PASSWORD	Contraseña del superusuario PostgreSQL	PostgreSQL container

4.4 Sistema de Ficheros y Volúmenes

Docker utiliza un sistema de ficheros por capas (overlay2). La imagen base de Python incluye el sistema de ficheros mínimo, y cada instrucción del Dockerfile agrega una capa inmutable. Al ejecutar el contenedor se añade una capa de escritura efímera que se descarta al detener el contenedor.

El volumen **postgres_data** monta un directorio del host dentro del contenedor de PostgreSQL en `/var/lib/postgresql/data`. Este mecanismo *bypasea* el sistema de ficheros efímero del contenedor y persiste los datos en el almacenamiento del host, sobreviviendo reinicios y recreaciones del contenedor.

4.5 Gestión de Señales y Ciclo de Vida de Procesos

Docker envía la señal **SIGTERM** al proceso PID 1 del contenedor cuando se ejecuta *docker compose down* o *docker stop*. Esto permite a los procesos realizar un cierre ordenado (*graceful shutdown*): PostgreSQL completa las transacciones en curso y sincroniza los buffers a disco; Nginx deja de aceptar nuevas conexiones y completa las activas. Tras un timeout configurable, Docker envía **SIGKILL** para forzar la terminación.

4.6 Comunicación entre Procesos (IPC)

La comunicación entre los tres servicios se realiza mediante **sockets TCP**, el mecanismo de IPC de red. Nginx abre un socket en el puerto 80, Flask en el 5000 y PostgreSQL en el 5432. La red interna Docker (*voting_net*) actúa como medio de comunicación entre estos procesos que corren en diferentes namespaces de red.

Internamente, psycopg2 (el driver Python para PostgreSQL) establece una conexión TCP al puerto 5432 del contenedor *db*. Cada petición HTTP abre una conexión nueva, realiza la operación y cierra la conexión en el bloque *finally*, implementando el patrón de conexión por demanda.

5. Componentes de Redes Aplicados

El proyecto aplica una variedad de conceptos de redes, desde el modelo TCP/IP hasta protocolos de capa de aplicación como HTTP y DNS. A continuación se describe cada componente de red presente en la solución.

5.1 Modelo TCP/IP y Pila de Protocolos

Todo el tráfico del sistema viaja sobre la pila de protocolos TCP/IP. A continuación se mapea cada capa del modelo con los componentes del proyecto:

Capa	Protocolo/Tecnología	Aplicación en el proyecto
Aplicación (L7)	HTTP/1.1, REST	API Flask (/votar, /resultados), JS fetch()
Aplicación (L7)	Nginx Proxy Protocol	Proxy de peticiones /api/* hacia Flask
Transporte (L4)	TCP	Sockets entre cliente-Nginx, Nginx-Flask, Flask-PostgreSQL
Red (L3)	IPv4 (172.x.x.x)	Red interna Docker voting_net (bridge)
Enlace (L2)	Ethernet virtual (veth pair)	Interfaz de red virtual de cada contenedor

5.2 DNS Interno de Docker

Docker Compose crea automáticamente un servidor DNS interno para la red *voting_net*. Cada servicio registra su nombre (*frontend*, *backend*, *db*) como hostname resolvable dentro de la red. Esto permite que Nginx use **proxy_pass http://backend:5000/** y que Flask use **host='db'** para conectarse a PostgreSQL, sin necesidad de conocer las IPs dinámicas asignadas por Docker.

Este mecanismo replica el comportamiento del DNS en redes reales: los servicios se identifican por nombre de dominio (FQDN dentro del namespace de la red) en lugar de direcciones IP, lo que facilita la reconfiguración y el escalado sin cambiar el código.

5.3 Protocolo HTTP/1.1 y Arquitectura REST

La API expone dos endpoints REST:

Método	Endpoint	Cuerpo / Respuesta	Código HTTP
POST	/votar	<pre>{"codigo": "EST001", "candidato_id": 1}</pre>	200 OK / 400 / 409 / 500
GET	/resultados	<pre>[{"id":1,"candidato":"Pavo Real","votos":5},...]</pre>	200 OK / 500

Las respuestas usan cabeceras HTTP estándar: **Content-Type: application/json** para indicar el tipo de contenido, y los códigos de estado HTTP para comunicar el resultado de la operación (200 éxito,

400 petición inválida, 409 conflicto, 500 error interno). Esto cumple con las restricciones REST de interfaz uniforme y comunicación sin estado.

5.4 Reverse Proxy con Nginx

Nginx actúa como **proxy inverso**, un patrón de red fundamental en sistemas distribuidos. Su configuración establece dos reglas de enrutamiento:

- **Archivos estáticos:** Las rutas que no coinciden con `/api/` son servidas directamente desde el directorio `/usr/share/nginx/html` usando la directiva `try_files`. Nginx los entrega directamente desde la caché del sistema operativo (page cache) sin invocar Python.
- **API REST:** Las rutas con prefijo `/api/` son redirigidas a `http://backend:5000/`. Nginx reescribe la URL (elimina el prefijo `/api/`) y agrega las cabeceras `X-Real-IP` y `Host` para que Flask conozca la IP real del cliente.

Este patrón implementa el concepto de **gateway de capa de aplicación**, análogo a un Load Balancer en entornos de producción. El cliente nunca interactúa directamente con Flask, lo que permite actualizar, reiniciar o escalar el backend sin interrumpir la conectividad del cliente.

5.5 Puertos, Sockets y Modelo Cliente-Servidor

El sistema implementa el modelo cliente-servidor en múltiples niveles:

Comunicación	Socket	Descripción
Navegador ↔ Nginx	TCP :8080	El navegador actúa como cliente HTTP; Nginx como servidor web.
Nginx ↔ Flask	TCP :5000	Nginx actúa como cliente HTTP upstream; Flask como servidor WSGI.
Flask ↔ PostgreSQL	TCP :5432	psycopg2 actúa como cliente de BD; PostgreSQL como servidor relacional.

5.6 Política de Mismo Origen (Same-Origin Policy)

Al servir tanto el frontend (HTML/JS) como la API desde el mismo origen (host:puerto, a través del proxy Nginx), se cumple con la política de mismo origen del navegador. Las peticiones `fetch()` del JavaScript se dirigen a `/api/votar` y `/api/resultados` como rutas relativas al mismo dominio, evitando la necesidad de configurar CORS (Cross-Origin Resource Sharing). Esto simplifica la configuración de seguridad y evita cabeceras adicionales en cada respuesta.

5.7 Red Bridge de Docker (voting_net)

Docker Compose crea una red bridge virtual llamada `voting_net`. Una red bridge es un switch virtual de capa 2 implementado en software por el kernel Linux mediante el módulo `bridge`. Docker asigna un segmento de red privada (ej. 172.20.0.0/16) y conecta las interfaces virtuales (veth pairs) de cada contenedor a este bridge. El router virtual del bridge gestiona el tráfico entre contenedores de la misma red y hacia el exterior mediante NAT (Network Address Translation) usando iptables.

6. Diseño de la Base de Datos

La base de datos PostgreSQL almacena la información del sistema de votación en un esquema relacional minimal de dos tablas, diseñado para garantizar la integridad referencial y la unicidad del voto.

Esquema de tablas:

Tabla	Columna	Tipo	Restricción	Descripción
candidatos	id	SERIAL	PRIMARY KEY	Identificador autoincremental
	nombre	VARCHAR(100)	NOT NULL	Nombre del candidato
votos	codigo	VARCHAR(20)	PRIMARY KEY	Código estudiantil — garantiza un voto por estudiante
	candidato_id	INT	FK → candidatos(id)	Referencia al candidato elegido
	timestamp	TIMESTAMPTZ	DEFAULT NOW()	Momento de emisión del voto (con zona horaria)

Consulta SQL de resultados:

El endpoint `/resultados` ejecuta la siguiente consulta que implementa un LEFT JOIN para incluir candidatos sin votos:

```
SELECT c.id, c.nombre, COUNT(v.codigo) AS votos FROM candidatos c
LEFT JOIN votos v ON c.id = v.candidato_id GROUP BY c.id, c.nombre
ORDER BY c.id;
```

El uso de **LEFT JOIN** garantiza que los tres candidatos siempre aparezcan en los resultados, incluso si no tienen votos (COUNT devuelve 0). **GROUP BY** agrega los votos por candidato. La columna `timestamp` con tipo **TIMESTAMPTZ** almacena la zona horaria del servidor, lo que permite auditorías temporales precisas.

7. Flujo de Datos y Comunicación entre Servicios

7.1 Flujo de Votación (POST /votar)

#	Componente	Acción
1	Navegador	El usuario ingresa su código y selecciona un candidato. Al hacer clic en 'Emitir voto', JavaScript ejecuta <code>fetch('POST /api/votar', body: JSON)</code> .
2	Nginx	Recibe la petición HTTP en el puerto 8080. La ruta coincide con <code>/api/</code> , por lo que reescribe la URL a <code>/votar</code> y hace <code>proxy_pass</code> a <code>http://backend:5000/votar</code> .
3	Flask	Recibe la petición en el blueprint <code>votar_bp</code> . Valida que el cuerpo sea JSON y que existan los campos <code>'codigo'</code> y <code>'candidato_id'</code> .
4	Flask → BD	Llama a <code>obtener_conexion()</code> para abrir un socket TCP al puerto 5432 del contenedor 'db'. Ejecuta <code>INSERT INTO votos(codigo, candidato_id) VALUES (%s,%s)</code> .
5	PostgreSQL	Verifica la PRIMARY KEY (<code>codigo</code>). Si ya existe → lanza <code>UniqueViolation</code> . Verifica FK (<code>candidato_id</code>). Si no existe → <code>ForeignKeyViolation</code> . Si es válido → inserta y confirma (<code>COMMIT</code>).
6	Flask	Captura las excepciones y construye la respuesta JSON apropiada (<code>{'exito': true/false, 'mensaje': '...'}</code>). Cierra la conexión en <code>finally</code> .
7	Nginx	Retransmite la respuesta JSON al navegador con las cabeceras HTTP originales de Flask.
8	Navegador	JavaScript analiza la respuesta. Si <code>exito:true</code> muestra alerta verde; si <code>false</code> muestra alerta roja con el mensaje de error.

7.2 Flujo de Consulta de Resultados (GET /resultados)

La página `resultados.html` ejecuta `cargarResultados()` al cargar y cada 5 segundos mediante `setInterval`. La función hace GET a `/api/resultados`. Nginx redirige a Flask, que ejecuta el LEFT JOIN en PostgreSQL y retorna un array JSON con `{id, candidato, votos}`. JavaScript calcula porcentajes, detecta el líder y renderiza dinámicamente las barras de progreso en el DOM.

8. Seguridad y Manejo de Errores

8.1 Inyección SQL — Consultas Parametrizadas

Todas las consultas SQL utilizan **parámetros posicionales** de psycopg2 (`%s`), nunca concatenación de cadenas. Esto previene ataques de inyección SQL: los valores del usuario se envían como parámetros separados al driver, que los escapa correctamente antes de construir la consulta. PostgreSQL trata los parámetros como datos, no como código SQL ejecutable.

8.2 Mínima Exposición de Puertos

Solo el puerto 8080 (Nginx) está expuesto al host. Los puertos 5000 (Flask) y 5432 (PostgreSQL) son accesibles únicamente dentro de la red Docker privada *voting_net*. Esto elimina la posibilidad de acceso directo a la API o a la base de datos desde fuera del host, reduciendo la superficie de ataque.

8.3 Validación de Entrada

El endpoint `/votar` valida que la petición sea JSON y que contengan los campos obligatorios antes de intentar cualquier operación de base de datos. Las respuestas de error nunca exponen trazas de pila (*stack traces*) ni mensajes internos del sistema: solo retornan mensajes de error genéricos en producción.

8.4 Manejo de Errores HTTP Semántico

Los códigos de estado HTTP transmiten semántica precisa: **200 OK** para operación exitosa, **400 Bad Request** para entradas inválidas, **409 Conflict** para voto duplicado (semánticamente correcto: el recurso ya existe) y **500 Internal Server Error** para errores inesperados. El frontend interpreta estos códigos para mostrar mensajes apropiados al usuario.

8.5 Gestión Segura de Credenciales

Las credenciales de base de datos se almacenan en el archivo `.env`, incluido en `.gitignore` para evitar su publicación accidental en el repositorio. El archivo `.env.example` provee una plantilla con valores de ejemplo, permitiendo que otros desarrolladores configuren su entorno sin acceder a las credenciales reales.

8.6 Cierre de Conexiones en Bloque Finally

Cada función del backend abre la conexión a PostgreSQL dentro de un bloque *try* y la cierra en el bloque *finally*, garantizando que la conexión se libere incluso si ocurre una excepción. Esto previene fugas de conexiones que agotarían el pool de conexiones disponibles de PostgreSQL con el tiempo.

9. Conclusiones

El desarrollo del Sistema de Votación Distribuido para la elección de la Mascota de la Facultad permitió aplicar de forma práctica e integrada los conceptos fundamentales de las asignaturas de Sistemas Operativos y Redes. A continuación se presentan las principales conclusiones derivadas del proceso:

1. Integración de conceptos teóricos en un sistema real:

La contenerización con Docker demostró de forma tangible cómo los namespaces del kernel Linux (PID, NET, MNT, UTS) proporcionan aislamiento real entre procesos, y cómo los cgroups limitan recursos. Lo que en teoría son abstracciones del SO, en la práctica se convirtieron en herramientas de diseño arquitectónico.

2. La arquitectura de red como pilar del sistema:

La decisión de usar Nginx como reverse proxy no fue únicamente técnica sino arquitectónica: unificó el punto de entrada, eliminó la necesidad de CORS, y demostró cómo el enrutamiento de red en capa 7 puede desacoplar servicios. Cada decisión de red (puertos expuestos, red bridge, DNS interno) tuvo implicaciones directas en la seguridad y la mantenibilidad del sistema.

3. La base de datos como garante de la integridad:

Delegar la regla de negocio 'un voto por código' al motor de base de datos mediante PRIMARY KEY fue una decisión correcta: las garantías ACID de PostgreSQL son más robustas que cualquier validación a nivel de aplicación frente a condiciones de carrera concurrentes.

4. Simplicidad y funcionalidad:

El sistema logró sus objetivos con una cantidad mínima de código y dependencias. Flask con dos rutas, Nginx con una configuración de 15 líneas y una base de datos con dos tablas fueron suficientes para implementar un sistema funcional, seguro y demostrable. La sobriedad técnica demostró que los fundamentos de SO y Redes son suficientes para construir sistemas reales.

5. Portabilidad y reproducibilidad:

La contenerización garantiza que el sistema funcione de forma idéntica en cualquier máquina con Docker instalado. Esta portabilidad es consecuencia directa de aplicar los conceptos de aislamiento de procesos y entornos del sistema operativo.